

what you need is a master reusable prompt that tells ChatGPT to:

1. Read the uploaded lab/task.
2. Understand every requirement.
3. Build a GitHub-ready workflow for completing the task.
4. Create all required README/documentation files.
5. Include commands/scripts/evidence folders where appropriate.
6. Package everything into a downloadable `.zip` file.
7. Make the structure reusable and professional.
8. Avoid inventing requirements that aren't in the uploaded task.

REUSABLE PROMPT: CREATE A GITHUB-READY LAB WORKFLOW ZIP

I have uploaded a lab/activity/task document for a Linux, cybersecurity, networking, programming, or technical practical assignment.

Your job is to analyze the uploaded task and create a **complete, professional, GitHub-ready project package** that I can download as a ZIP file and use while performing the activity.

1. ANALYZE THE UPLOADED TASK

First, carefully read the entire uploaded task.

Identify:

- The objective
- Every task/step
- Required commands
- Required files
- Required directories
- Required configurations

- Required screenshots
- Required evidence
- Submission requirements
- Expected final results
- Any restrictions or special instructions

Do NOT invent requirements that are not supported by the uploaded task.

Use the exact terminology, usernames, filenames, paths, commands, and requirements from the activity wherever applicable.

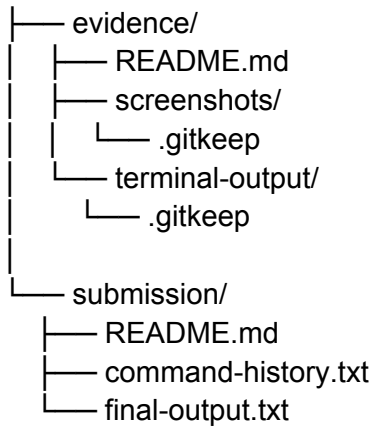
2. CREATE A PROFESSIONAL GITHUB PROJECT

Create a complete GitHub-ready project specifically for this activity.

Use a professional project name based on the uploaded task.

The project should have an organized structure similar to:

```
project-name/
├── README.md
├── .gitignore
├── docs/
│   ├── README.md
│   └── task-overview.md
├── workflow/
│   ├── README.md
│   └── workflow.md
├── commands/
│   ├── README.md
│   └── commands.sh
├── scripts/
│   ├── README.md
│   └── [relevant scripts]
```



Modify this structure when necessary.

Do NOT create unnecessary files or folders just for the sake of making the project larger.

The structure should reflect the actual requirements of the uploaded activity.

3. CREATE THE MAIN README.md

Create a professional **README.md** containing:

- Project title
- Project description
- Objective
- Lab/activity requirements
- Technologies/tools used
- Prerequisites
- Environment/setup information
- Project structure
- Step-by-step workflow
- Commands used
- Expected results
- Evidence requirements
- Submission requirements
- Troubleshooting section where useful
- Safety/permission notes where applicable

The README should be written so that another student could understand how to perform the activity from the repository.

4. CREATE A DETAILED WORKFLOW

Create:

workflow/workflow.md

This should contain the complete practical procedure.

Break the activity into logical phases.

For example:

Phase 1 — Environment Preparation
Phase 2 — User/Account Configuration
Phase 3 — Directory/File Creation
Phase 4 — Configuration
Phase 5 — Verification
Phase 6 — Error Testing/Recovery
Phase 7 — Evidence Collection
Phase 8 — Final Submission

Only use phases that make sense for the uploaded activity.

For every step include:

- What to do
- Command to run
- Expected result
- Verification command
- What evidence/screenshot to capture
- Any important notes

Clearly identify commands that must be executed as:

root
sudo
normal user
specific user

when applicable.

5. CREATE A COMMANDS FILE

Create:

commands/commands.sh

containing the commands needed to complete the activity in the correct order.

Add comments explaining important commands.

Example:

```
# Create the required user  
sudo useradd ...
```

```
# Switch to the required user  
su - ...
```

```
# Create the workspace  
mkdir -p ...
```

```
# Verify the result  
ls -la ...
```

Do not blindly put every command into one script if executing the commands automatically could be unsafe or inappropriate.

If some commands require manual interaction, clearly mark them.

If commands must be executed by different users, separate them clearly.

6. CREATE SCRIPTS ONLY WHEN APPROPRIATE

If the task can benefit from executable scripts, create them under:

scripts/

For example:

```
scripts/setup.sh  
scripts/verify.sh  
scripts/cleanup.sh
```

Only create scripts when they are genuinely useful for the activity.

Do not automate steps that should intentionally be performed manually for learning, assessment, verification, or security reasons.

Each script must include:

- Comments
- Clear purpose
- Safe command ordering
- Error handling where appropriate

Also create:

```
scripts/README.md
```

explaining what each script does and how to use it.

7. CREATE EVIDENCE STRUCTURE

Create an organized evidence directory.

For example:

```
evidence/  
├── README.md  
├── screenshots/  
│   ├── .gitkeep  
└── terminal-output/  
    ├── .gitkeep
```

The evidence README must explain:

- What screenshots are required

- What commands/output should be captured
- When to capture each screenshot
- Suggested screenshot filenames
- What each screenshot proves

Use meaningful filenames such as:

01-user-created.png

02-workspace-created.png

03-file-permissions.png

04-final-directory-structure.png

Do not create fake screenshots or fake evidence.

8. CREATE SUBMISSION MATERIALS

Create:

submission/README.md

Explain exactly what the student needs to submit according to the uploaded task.

Where appropriate, provide templates/placeholders such as:

submission/

|— README.md

|— command-history.txt

|— final-output.txt

Do not fabricate completed results.

Use placeholders such as:

[PASTE COMMAND HISTORY HERE]

when actual results must be produced by me while performing the lab.

9. CREATE .gitignore

Create a professional `.gitignore` appropriate for the project.

It should prevent unnecessary files from being committed, such as:

```
.DS_Store  
Thumbs.db  
*.log  
*.tmp  
*.swp  
.vscode/  
.idea/
```

Add other appropriate exclusions based on the activity.

Do NOT ignore files that are explicitly required as evidence or submission materials.

10. MAKE THE WORKFLOW LEARNING-FRIENDLY

The project should not merely give me commands.

For important commands, explain:

- What the command does
- Why it is required
- What I should expect
- How to verify it
- What evidence to capture

For example:

Command:

```
ls -la
```

Purpose:

Displays all files, including hidden files, together with permissions, ownership, and other metadata.

Verification:

Confirm that the expected files and permissions are displayed.

Evidence:

Capture a screenshot showing the command and its output.

11. HANDLE USER/PERMISSION REQUIREMENTS CAREFULLY

If the task involves:

- root
- sudo
- users
- groups
- file ownership
- file permissions
- privileged commands

clearly indicate which commands require elevated privileges and which must be performed as the specified normal user.

Never silently change the task's permission requirements.

For security-related activities, preserve the educational purpose of the task.

12. INCLUDE A FINAL VERIFICATION CHECKLIST

Create a checklist such as:

Final Verification Checklist

- [] All required users created
- [] Required directories created

- [] Required files created
- [] Required permissions configured
- [] Required ownership configured
- [] Required commands executed
- [] Required error/recovery test completed
- [] Final output verified
- [] Screenshots captured
- [] Command history saved
- [] Submission requirements completed

Customize the checklist to the actual uploaded task.

13. DO NOT FABRICATE RESULTS

This is very important.

You are creating a workflow/template for me to perform the activity.

Therefore:

- Do not pretend I have already completed the lab.
- Do not create fake command output.
- Do not create fake screenshots.
- Do not claim that commands were successfully executed.
- Do not fabricate evidence.
- Do not invent usernames, results, permissions, IP addresses, hashes, or configurations unless the task explicitly provides them.

Use placeholders where my actual results are required.

14. VALIDATE THE PROJECT

Before creating the ZIP:

Check that:

- README files are complete.
- Commands correspond to the uploaded task.

- Workflow steps are logically ordered.
- Paths and filenames are consistent.
- No required task step has been omitted.
- No fake evidence has been included.
- Scripts are syntactically reasonable.
- `.gitignore` is present.
- Evidence folders are present.
- Submission requirements are represented.
- The project can be uploaded directly to GitHub.

15. CREATE THE DOWNLOADABLE ZIP

After creating all files, package the entire project into a ZIP archive.

The ZIP should contain ONE top-level project directory:

`project-name/`

and everything should be inside it.

Example:

```
project-name.zip
└─ project-name/
    ├── README.md
    ├── .gitignore
    ├── docs/
    ├── workflow/
    ├── commands/
    ├── scripts/
    ├── evidence/
    └─ submission/
```

Do not put unrelated files into the ZIP.

16. FINAL RESPONSE

After generating the project:

1. Give me the downloadable ZIP file.
2. Tell me the project name.
3. Briefly show the major folders/files included.
4. Tell me what I should do first after downloading it.
5. Do not make me manually recreate the files.

The primary deliverable must be the **downloadable ZIP file**, not merely text describing what the ZIP should contain.

IMPORTANT

Treat every uploaded task as a separate project.

For each new activity I upload, repeat this process from the beginning and customize the entire GitHub project to that specific activity.

Do not reuse commands, filenames, users, paths, or requirements from a previous lab unless they are also present in the new uploaded task.